Tom_Casey_CSE511_Hot_Spot_Analysis_Project_Report.pdf

1. Reflection

My approach to the CSE 511 Hot Spot Analysis Project emphasized careful specification reading, incremental implementation, and repeated correctness validation across all provided test cases. Because the project consisted of two distinct components, Hot Zone Analysis and Hot Cell Analysis, I planned my workflow to handle each part independently before integrating them into a single Spark SQL pipeline.

I began with the Hot Zone Analysis component by focusing solely on the ST_Contains spatial predicate. I studied the problem description, lecture material, and the assignment examples to understand how rectangular spatial ranges and point coordinates should be processed. To avoid off-by-one errors and incorrectly flipped coordinate boundaries, I normalized rectangle coordinates by converting them into minimum and maximum values before testing containment. This step significantly reduced edge-case errors and ensured that the point-in-rectangle logic behaved consistently across the entire dataset.

For the Hot Cell Analysis portion, I took a more iterative approach. Before writing the full solution, I first verified that the coordinate transformation functions in HotcellUtils.scala produced correct X, Y, and Z grid values for sample inputs. After validating X/Y/Z computation and bounding-box filtering, I implemented the group-by aggregation to count taxi pickups per cell. Once the dataset was correctly transformed into a 3D grid, I worked through the Getis–Ord G* statistic step-by-step: computing global mean and standard deviation, identifying cell neighbors, summing neighbor values, and determining the correct number of neighbors for each cell based on boundary constraints.

A major portion of the debugging process involved validating the ordering and structure of the output. Across multiple attempts, I noticed that my intermediate outputs matched expected values but appeared in slightly different orders. By refining my final sort criteria, sorting in descending order by G-score, I ensured deterministic output across all test cases.

Altogether, the project required a combination of spatial reasoning, SQL-based aggregation, mathematical correctness, and careful debugging of distributed Spark transformations. Working sequentially, testing intermediate stages, and validating outputs against sample test cases allowed me to complete the assignment successfully.

2. Lessons Learned

Completing the Hot Spot Analysis Project strengthened my understanding of spatial computation, distributed data processing, and statistical hot spot detection. One of the most important lessons I learned is the role of data normalization and coordinate transformation. Incorrectly normalized coordinates or incorrect boundary calculations can propagate errors into

subsequent group-by aggregations and lead to incorrect z-score evaluation. Ensuring consistent coordinate handling proved essential for correctness.

A second major lesson involved understanding how Spark SQL executes computational pipelines. The assignment required multiple chained transformations, including UDFs, filters, group-by aggregations, joins, and statistical computations. Initially, it was not obvious where errors originated, but by checking data immediately after each transformation, I became more comfortable with Spark's lazy evaluation model and how SQL queries translate into distributed execution plans.

Another important insight came from studying the Getis–Ord G* statistic itself. Before this project, I had not implemented spatial autocorrelation measures. Breaking the formula into parts, such as neighbor summation, neighbor count determination, global mean, variance, and standard deviation, helped me understand how G-scores highlight statistically significant clustering patterns. The exercise clarified why hot spot analysis is widely used in spatial analytics, especially in fields such as transportation, crime mapping, and public health.

Finally, I learned the importance of validating intermediate outputs and not relying solely on final results. Debug printing specific spatial cells, verifying neighbor counts at boundaries, and calculating sample mean and variance strengthened my confidence in the correctness of the implementation. This project reaffirmed that distributed systems benefit greatly from incremental verification and small-scale deterministic testing.

3. Implementation

3a. Hot Zone Analysis: Understanding ST Contains

The Hot Zone Analysis component performs a spatial join between a set of rectangular regions and taxi pickup points. The function: def ST_Contains(queryRectangle: String, pointString: String) returns true if a point lies inside a rectangle and false otherwise.

Implementation breakdown:

1. The rectangle string contains four comma-separated coordinates:

(x1, y1, x2, y2) representing the two opposite corners of the rectangle.

2. To handle flipped or unordered input, the implementation computes:

- xMin = min(x1, x2)
- xMax = max(x1, x2)
- yMin = min(y1, y2)
- yMax = max(y1, y2)

3. The point string contains the two comma-separated coordinates: (x, y).

4. The function returns true if and only if: xMin ≤ x ≤ xMax and yMin ≤ y ≤ yMax.

5. Spark SQL registers ST_Contains as a UDF and applies it during a join, allowing each point to be matched with all rectangles that contain it.

This component ensures that the project can correctly determine which taxi pickups fall within specific Manhattan grid rectangles.

3b. Hot Cell Analysis: Steps Required Before Z-Score Calculation

Hot Cell Analysis extends the Hot Zone concept to three dimensions (longitude, latitude, and day-of-month), enabling identification of spatio-temporal hot spots based on taxi data density. Several steps must be completed before computing each cell's Getis–Ord G* z-score.

Step 1: Convert Raw Data Into X/Y/Z Grid Coordinates

HotcellUtils.scala provides three UDFs:

- CalculateX(pickupLongitude)
- CalculateY(pickupLatitude)
- CalculateZ(pickupTimestamp)

These map real-world coordinates into discrete grid cells using the instructor-specified spatial resolution (coordinateStep).

Z corresponds to the day of the month (1–31).

Step 2: Apply Spatial-Temporal Boundaries

Cells are filtered using the assignment-provided Manhattan bounding box limits:

- minX ≤ x ≤ maxX
- minY ≤ y ≤ maxY
- 1 ≤ z ≤ 31

This ensures only Manhattan-area taxi pickups remain.

Step 3: Aggregate Cell Counts

Spark groups points by grid cell:

GROUP BY x, y, z

COUNT(*) AS cellValue

This produces a (x, y, z → count) table representing pickup density per 3D cell.

Step 4: Compute Global Statistics

Before computing z-scores, Spark must calculate dataset-wide values:

- Total number of possible cells:
- totalCells = (maxX−minX+1)(maxY−minY+1)(maxZ−minZ+1)
- sumX = Σ(count)
- sumX2 = Σ(count^2)
- mean = sumX / totalCells
- stdDev = sqrt((sumX2 / totalCells) − mean^2)

These statistics provide the normalization needed for Getis–Ord G*.

Step 5: Neighbor Identification via Self-Join

Each cell must sum the values of all neighbors where:

$|x1 - x2| \leq 1$

$|y1 - y2| \leq 1$

$|z1 - z2| \leq 1$

This produces up to 27 neighbors (3×3×3 cube) per cell.

Step 6: Determine Neighbor Count

Using another UDF, the implementation counts how many neighbors a cell should have based on whether it lies at:

- an interior location,
- an edge,
- a corner,
- or a temporal boundary (day 1 or 31).

This step is essential for accurate denominator computation in the G* formula.

Step 7: Compute G-Score (Getis–Ord G*)

For each cell:

G = (sumNeighbors − mean * numNeighbors) /

(stdDev * sqrt((numNeighbors * totalCells − numNeighbors^2) / (totalCells − 1)))

A high G-score indicates significant clustering, that is, a hot cell.

Step 8: Output the Top Hot Cells

Finally, results are sorted by:

1. G-score (descending)
2. X coordinate
3. Y coordinate
4. Z coordinate

Only the top hot cells (typically 50) are returned.

Conclusion

This project provided practical experience with spatial computation, Spark SQL data processing, and statistical hot spot detection. Through iterative development and verification, I successfully implemented all required components of Hot Zone and Hot Cell Analysis, including spatial containment logic, coordinate transformation, neighbor evaluation, and Getis–Ord G* score computation. The process strengthened my understanding of spatial analytics and distributed computation while reinforcing the importance of incremental debugging, numerical correctness, and systematic pipeline design.